

Author

Akshata Kumar
22f3001688
22f3001688@ds.study.iitm.ac.in

Introduction

I am a diploma level student in IITM BS degree. I am passionate about Machine Learning, Data Science and Application Development. Through the development of the Trekking Management Application as part of my MAD-2 project, I gained hands-on experience in designing and implementing a full-stack web application with multiple user roles and real-world workflow requirements. The project helped me understand how to structure a scalable backend using Flask APIs, design relational database schemas using SQLite and SQLAlchemy ORM, and build interactive front-end interfaces using VueJS and Bootstrap. I also learned how to implement authentication and role-based access control using JWT tokens to ensure secure access for Admin, Company, and Student users. Additionally, the integration of Redis caching and Celery background jobs introduced me to performance optimization and asynchronous task processing for features such as reminders, report generation, and CSV exports. Overall, this project strengthened my understanding of backend architecture, RESTful API design, frontend integration, and the development of scalable web applications.

Description

TrekScape (my platform's name) is a role-based Trekking Management Application that enables Admins, Trek Staff, and Trekkers to efficiently manage trekking activities. The system provides secure authentication, trek management, booking, participant tracking, and historical records while preventing overbooking and unauthorized access. It also incorporates Redis caching and Celery-based background jobs for reminders, reports, and CSV exports to improve performance and automation.

AI/LLM Usage: ~20%

AI (ChatGPT and Claude) was used mostly for debugging, Flask routing, and Celery-based background jobs help.

Technologies used

Backend:

- Flask
- SQLite
- SQLAlchemy ORM
- Redis
- Celery
- JWT Authentication (Flask-JWT)

Frontend:

- VueJS
- Bootstrap
- Axios

DB Schema Design

The TrekScope application uses a **relational SQLite database** created programmatically through SQLAlchemy models. A unified User model is used to represent all three roles (Admin, Trek Staff, and Trekker), ensuring a normalized database design and reducing redundancy. The schema is designed to maintain data integrity, support role-based access control, and efficiently manage trekking operations.

User Table

Column	Data Type	Constraints
id	Integer	Primary Key, Auto Increment
full_name	String(150)	NOT NULL
email	String(255)	UNIQUE, NOT NULL
password_hash	String(255)	NOT NULL
phone	String(20)	Nullable
role	String(50)	NOT NULL (admin/staff/trekker)
is_active	Boolean	Default=True
is_blacklisted	Boolean	Default=False
created_at	DateTime	Default=Current Timestamp

- One User can have many Bookings.
- One Staff User has one Staff Profile.
- One Staff User can manage multiple Treks.

Staff Profile Table

Column	Data Type	Constraints
id	Integer	Primary Key
user_id	Integer	Foreign Key → users.id, UNIQUE
experience_years	Integer	Nullable
specialization	String(150)	Nullable
emergency_contact	String(20)	Nullable
status	String(50)	Default='active'
created_at	DateTime	Default Timestamp

Trek Table

Column	Data Type	Constraints
id	Integer	Primary Key
trek_name	String(150)	NOT NULL
location	String(150)	NOT NULL
description	Text	Nullable
difficulty	String(20)	NOT NULL
duration_days	Integer	NOT NULL
total_slots	Integer	NOT NULL
available_slots	Integer	NOT NULL
start_date	Date	NOT NULL
end_date	Date	NOT NULL
status	String(50)	NOT NULL
assigned_staff_id	Integer	Foreign Key → users.id
created_at	DateTime	Default Timestamp

- One Staff can manage multiple Treks.
- One Trek can have multiple Bookings.

Booking Table

Column	Data Type	Constraints
id	Integer	Primary Key
user_id	Integer	Foreign Key → users.id
trek_id	Integer	Foreign Key → treks.id
booking_date	DateTime	Default Timestamp
status	String(50)	NOT NULL
payment_status	String(50)	Default='not_applicable'
completed_date	DateTime	Nullable
created_at	DateTime	Default Timestamp

- Many Bookings belong to one User.
- Many Bookings belong to one Trek.

Booking History Export Table

Column	Data Type	Constraints
id	Integer	Primary Key
task_id	String	UNIQUE, NOT NULL
user_id	Integer	Foreign Key → users.id
filename	String	Nullable
status	String	NOT NULL
created_at	DateTime	Default Timestamp
completed_at	DateTime	Nullable

Design Rationale

- 1) A **single User table** is used with a 'role' attribute to support Admin, Trek Staff, and Trekker accounts without duplicating authentication data.
- 2) A separate **StaffProfile** table stores staff-specific information, maintaining database normalization and allowing easy extension of staff details.
- 3) The **Trek** table stores all trek-related information, while the **assigned_staff_id** foreign key establishes a one-to-many relationship between staff and treks.
- 4) The **Booking** table acts as a junction table between Users and Treks, enabling many-to-many relationships while maintaining complete booking history.
- 5) A **Unique Constraint** on (**user_id**, **trek_id**) prevents duplicate bookings for the same trek.
- 6) The **BookingHistoryExport** table securely tracks asynchronous CSV export tasks, ensuring proper ownership verification and preventing unauthorized access to exported files.
- 7) Indexes on frequently queried fields such as **status** and **difficulty** improve search performance and complement the Redis caching layer implemented in the application.
- 8) The schema is normalized, minimizes redundancy, enforces referential integrity through foreign keys, and supports scalable role-based operations required by the TrekScape application.

API Design

The major API modules include:

- **Authentication APIs:** User registration, login, profile retrieval, and profile update.
- **Admin APIs:** Dashboard statistics, trek management (create, update, delete), staff management, staff assignment, user management, booking management, and historical trekking records.
- **Trek Staff APIs:** Assigned treks, participant management, trek status updates, slot management, and trek completion.
- **Trekker APIs:** Browse/search/filter treks, trek booking, booking cancellation, booking history, profile management, and CSV export.
- **Background Job APIs:** Trigger asynchronous booking history export and check export status using Celery tasks.
- **Caching:** Frequently accessed trek listing APIs are optimized using Redis caching with automatic cache invalidation after trek or booking updates.

The API follows standard REST principles by using appropriate HTTP methods (**GET**, **POST**, **PUT**, **DELETE**) and returns consistent JSON responses along with suitable HTTP status codes. All API endpoints are documented separately using an **OpenAPI (YAML)** specification.

System Architecture

The TrekScape application is developed using the **Model-View-Controller (MVC)** architecture to ensure modularity, maintainability, and separation of concerns. The **Flask** backend exposes REST APIs, while the **Vue.js** frontend provides a responsive single-page user interface. Bootstrap 5 is used for styling and responsive layouts.

The backend is organized into dedicated folders for different responsibilities. The **models** folder contains SQLAlchemy ORM models representing the database schema. The **controllers** handle incoming HTTP requests and responses, while the **services** folder contains the core business logic, ensuring that controllers remain lightweight. API endpoints are organized under the **routes** folder, and reusable utilities such as constants and helper functions are placed inside the **utils** folder. Background tasks such as CSV export, reminder emails, and monthly reports are implemented in the **tasks** folder using Celery. Application configuration is managed through **config.py**, while **extensions.py** initializes Flask extensions such as SQLAlchemy, JWT, CORS, Redis, and Celery.

The frontend follows Vue.js component-based architecture. The **views** folder contains complete application pages, **components** contains reusable UI components, **router** manages client-side navigation, **services** handles API communication using Axios, and **store** maintains shared application state. Authentication is implemented using **JWT tokens**, and role-based routing ensures that users are redirected to the appropriate dashboard after login.

Features Implemented

- User authentication with JWT-based login and role-based access control.
- Pre-seeded Admin account created programmatically.
- Trek Staff creation and management by Admin.
- Trek creation, update, staff assignment, and management.
- Trek browsing, searching, filtering, and booking for Trekkers.
- Prevention of duplicate bookings and overbooking beyond available slots.
- Staff management of assigned treks, participant lists, trek status, and available slots.
- Complete booking history and trekking history management.
- Admin dashboards with statistics and complete historical booking records.
- Redis caching for frequently accessed trek listing APIs with automatic cache invalidation.
- Celery and Redis integration for asynchronous background processing.
- User-triggered asynchronous CSV export of trekking history.
- Daily reminder emails for upcoming treks.
- Monthly HTML activity report emailed to the Admin.
- Secure role-based authorization throughout the application.

Video

 Video Project.mp4